

Methods for Detecting Cyber Attacks – Conclusionary Practice

Eitan Beriy and Shachar Tzarfati

November 2025

1 Abstract

Package managers have become an integral part of modern software development, enabling developers to share code, reduce duplication, and simplify the build process. However, attackers increasingly exploit these ecosystems—particularly npm—to distribute malicious packages, posing significant security risks. This research aims to develop a machine learning-based detection tool that identifies malicious behavior in npm packages by analyzing code structure, metadata, and dependency relationships. The proposed approach combines static and behavioral features to distinguish between benign and malicious packages. The expected outcome is a scalable and accurate detection model that can assist developers and security platforms in mitigating supply-chain attacks within the npm ecosystem.

2 Introduction

Package managers have become a cornerstone of modern software development. They enable developers to easily share, reuse, and update code across projects, significantly reducing development time and improving productivity. Among these, npm (Node Package Manager) stands out as one of the largest software registries in the world, hosting millions of open-source JavaScript packages that power web applications, servers, and tools used by developers globally. This extensive ecosystem, while valuable, relies heavily on community trust and automation in managing dependencies. Unfortunately, the same openness that makes npm powerful also exposes it to significant security risks. Attackers have repeatedly exploited npm to distribute malicious or compromised packages, using techniques such as typosquatting, dependency confusion, account takeovers, and malicious postinstall scripts. These attacks allow threat actors to steal credentials, exfiltrate data, install backdoors, or execute arbitrary code on developers' machines and production servers. The sheer number of packages, the depth of dependency chains, and the speed of automatic updates make it easy for such threats to spread unnoticed. Compromises in npm have a cascading

impact across the entire software supply chain. A single malicious package can infect thousands of downstream projects, many of which are integral to web services, enterprise systems, and public infrastructure. High-profile incidents—such as the event-stream and ua-parser-js attacks—demonstrate how easily trust in open-source ecosystems can be abused. As organizations increasingly depend on npm, ensuring its security has become critical to maintaining the integrity and reliability of modern software. Detecting malicious npm packages remains difficult because traditional security measures—like manual reviews and signature-based detection—do not scale to the ecosystem’s size or adapt quickly to new attack techniques. Malicious actors often disguise harmful code within legitimate functionality, obfuscate payloads, or exploit trusted dependencies to evade detection. Building an effective defense therefore requires automated, intelligent approaches that can recognize patterns of malicious behavior without relying solely on known signatures.

3 Literature Review

Zimmermann et al. [?] present the Backstabber’s Knife Collection, a dataset and analysis of 174 real-world malicious packages distributed via npm, PyPI, and RubyGems, establishing the prevalence of supply-chain attacks in open-source ecosystems. *Strengths*: Comprehensive dataset of real-world malicious packages; well-grounded and reproducible. *Limitations*: Small dataset; descriptive only; limited metadata analysis.

Neupane et al. [?] provide the first comprehensive taxonomy of package confusion attacks, identifying mechanisms beyond typosquatting and demonstrating how these can inform practical detection strategies. *Strengths*: Clear taxonomy of confusion attacks; strong theoretical insight. *Limitations*: Descriptive, no automated tool; narrow focus; limited large-scale evaluation.

In follow-up work, Neupane et al. [?] introduce ConfuGuard, a state-of-the-art detector for confusion attacks that reduces false positives through metadata-driven filtering at the cost of additional analysis latency. *Strengths*: Practical tool for confusion attacks; good accuracy using metadata. *Limitations*: Only detects confusion attacks; adds latency; no zero-day coverage.

Huang et al. [?] propose DONAPI, a behavior-sequence-based detector built from a synchronized cache of over three million npm packages, identifying 325 malicious samples and discovering new API sequences unseen in existing malware. *Strengths*: Deep behavioral analysis; hybrid static + dynamic detection; finds unknown malware. *Limitations*: High infrastructure cost; focuses on API calls; may not suit real-world CI/CD pipelines.

Huang et al. [?] further extend this line of work with Cerebro, a cross-ecosystem malicious-package detector that achieves high accuracy on both npm and PyPI and leads to the discovery of hundreds of new malicious packages. *Strengths*: Cross-ecosystem detection; discovers hundreds of new malware samples. *Limitations*: Heavy data and code-processing requirements; limited discussion of false positives.

Table 1: Comparison Table

Paper	Problem	Features	Method / Model	Dataset	Results
Buckstaber’s Knife Collection	Lack of systematic analysis of real malicious packages	Metadata, behavior, payload triggers	Manual taxonomy	174 malicious packages	61% typoquatting; 21 clusters
Beyond Typoquatting	Semantic confusion imports	11 confusion patterns	Semantic similarity engine	1,232 attacks + 1.8M names	36% confusing pairs
ConfuGuard: Using Metadata to Detect Active and Stealthy Package Confusion Attacks	Metadata-based confusion evasion	Maintainer/README/domain features	Metadata rules + embeddings NeopaceDB + ConfuDB	FP 80%→28%;	630 attacks
RONAPI: Malicious NPM Packages Detector Using Behavior Sequence Knowledge	Static trade fall on obfuscation	96 APIs, dynamic sequences	Static + dynamic behavior analysis	1,129 malicious + 3,000 benign	F1=95.1%; 325 new malware
Trilling Two Birds with One Stone: Malicious Package and Vulnerability Detection in Software Supply Chains	No cross-ecosystem model	16 static features, behavior sequences	LLM classifier	2,675 malicious + 7,391 benign	+10% precision; 1,482 malwares
Leveraging Large Language Models to Detect npm Malicious Packages	Obfuscation defeats rules	Semantic + metadata prompts	Instruction-tuned LLM	12,000 packages	92-96% accuracy
MalPacDetector: An LLM-based Malicious NPM Package Detector	LLM hallucination	Multi-stage prompts	LLM with prompt engineering	7,700 packages	93-97% accuracy; -80% FP
SmallWorld with High Risks: A Preliminary Study of Security Threats in the open Ecosystem	Frugal dependency graph	Graph metrics, maintainer influence	Network analysis	1.2M packages	Top 1% affect 55% ecosystem
Towards Measuring Supply Chain Attacks on Language-Based Package Ecosystems	No mislead detection	Metadata, static, dynamic	MALOSS pipeline	1.3M packages	339 new malwares
What Are Weak Links in the open Supply Chain? A Study of Metadata, Maintainers, and Dependency Graphs	Maintainer/package weaknesses	6 weak-link signals	Metadata + survey	1.63M packages	8,494 vulnerable packages

Zhang et al. [?] evaluate the use of large language models for detecting malicious npm packages and introduce SocketAI, showing that GPT-based analysis significantly outperforms traditional static detectors while remaining computationally feasible through pre-filtering. *Strengths*: Innovative LLM-based detection; very high precision; effective hybrid workflow. *Limitations*: Expensive LLM APIs; metadata and maintainer analysis not considered; risk of hallucinations.

Li et al. [?] propose MalPacDetector, an LLM-driven feature-generation system that achieves state-of-the-art false-positive and false-negative rates and identifies 39 previously unknown malicious packages confirmed by the npm security team. *Strengths*: Automated feature discovery; strong real-world impact; finds previously unknown malware. *Limitations*: Strong dependency on LLMs; black-box features; scalability concerns.

Zeng et al. [?] analyze systemic weaknesses in the npm ecosystem, highlighting risks such as dependency concentration, unmaintained packages, and maintainers with excessive control. *Strengths*: Strong structural analysis; valuable ecosystem-wide insights. *Limitations*: No malicious-package detection; lack of dataset; risk signals not operationalized.

Duan et al. [?] introduce MALOSS, a measurement pipeline for supply-chain attacks across multiple package managers, discovering 339 new malicious packages and demonstrating the challenges of program analysis for dynamic languages. *Strengths*: Large-scale discovery of new malware; demonstrates practical pipelines. *Limitations*: Focused largely on PyPI/RubyGems; dynamic analysis slow and expensive.

Tenny et al. [?] analyze 1.63 million npm packages and identify six “weak-link” metadata signals — such as install scripts and expired maintainer-domain emails — that correlate with elevated security risk. *Strengths*: Massive-scale metadata analysis; practical and actionable risk indicators. *Limitations*: No end-to-end classifier; possible false positives; limited empirical validation.

4 Methodology

The methodology of this research is based on analyzing metadata features extracted from the manifest.json files of npm packages included in the DataDog

malicious packages dataset, and on building a classification model capable of detecting malicious behavior using metadata signals alone. The methodology consists of four main stages:

1. Data Preprocessing:

- Extract relevant features from the dataset, including metadata (e.g., package name, version, dependencies) and optional static indicators (e.g., scripts, file counts).
- Clean and normalize the data, handling missing values and encoding categorical features.

2. Feature Engineering:

- Construct features that capture package risk signals, such as unusual naming patterns, dependency anomalies, and maintainers' histories.
- Optionally, create derived features (e.g., number of dependencies, presence of install scripts) to improve model discrimination.

3. Model Selection and Training:

- Evaluate multiple classification models, such as Random Forests, Gradient Boosted Trees, and Neural Networks.
- Use cross-validation to tune hyperparameters and avoid overfitting.

4. Evaluation:

- Assess model performance using metrics like precision, recall, F1-score, and ROC-AUC.
- Analyze false positives and false negatives to understand limitations and improve feature design.

5 Dataset

The dataset used in this research is the Malicious Software Packages Dataset published by DataDog. It contains hundreds of real malicious npm packages that were identified, verified, and documented by professional security researchers. Each package in the dataset is represented by a manifest.json file, which includes essential metadata such as the package name, version history, dependency list, maintainers, timestamps, and descriptive fields. The dataset focuses specifically on real-world software supply-chain attacks, including typo squatting, dependency confusion, malicious installation scripts, and unauthorized package publication. For this research, only the metadata contained in each manifest.json file was used as input for analysis and feature extraction, without examining the package code itself.

Link: <https://github.com/DataDog/malicious-software-packages-dataset/tree/main>

References

1. Zimmermann, T., Staicu, C. A., Tenny, C., & Pradel, M. (2019). *Backstabber's Knife Collection: A review of open-source software supply-chain attacks*. IEEE Symposium on Security and Privacy (S&P).
2. Neupane, S., Holmes, G., Wyss, E., Davidson, D., & De Carli, L. (2022). *Beyond typosquatting: An in-depth look at package confusion attacks*. USENIX Security Symposium.
3. Neupane, S., Holmes, G., Wyss, E., Davidson, D., & De Carli, L. (2023). *ConfuGuard: Detecting active and stealthy package confusion attacks*. ACM Conference on Computer and Communications Security (CCS).
4. Huang, C., Wang, N., Wang, Z., Sun, S., Li, L., et al. (2023). *DON-API: Detecting malicious npm packages using behavior-sequence knowledge*. IEEE Security and Privacy Workshops (SPW).
5. Huang, C., Wang, N., Wang, Z., Li, L., Chen, J., et al. (2023). *Cross-ecosystem malicious package detection using behavior sequences*. ACM Conference on Computer and Communications Security (CCS).
6. Zhang, Y., Liu, H., Wang, X., & Chen, K. (2023). *Leveraging large language models to detect npm malicious packages*. arXiv preprint arXiv:230X.XXXXX.
7. Li, F., Wang, X., & Hou, J. (2023). *MalPacDetector: LLM-based malicious npm package detection*. arXiv preprint arXiv:230X.XXXXX.
8. Zeng, Y., Wu, Z., Wang, T., & Shi, L. (2022). *SmallWorld with high risks: Security threats in the npm ecosystem*. IEEE Transactions on Software Engineering (TSE).
9. Duan, R., Alrawi, O., Kasturi, R. P., Elder, R., Saltaformaggio, B., & Lee, W. (2021). *MALOSS: Measuring supply-chain attacks on package managers*. USENIX Security Symposium.
10. Tenny, C., Zimmermann, T., Staicu, C. A., & Pradel, M. (2023). *Weak links in the npm supply chain*. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE).